

Estructuras de Datos, Strings e Iteración

Profesor: Pablo J Piccoli Bornia

Auxiliares: Daniela Longas - M Victoria Tolosa

TUPLAS



Tuplas: Conceptos Básicos

Idea: Estructura para almacenar secuencias fijas de valores.

Característica principal: Son **INMUTABLES** (no se pueden modificar).

Operaciones: Crear con una tupla (a, b), saber cuántos elementos tiene una tupla con len(), acceder a los elementos con [].

Ejemplo:

```
t = ('hola', 1, 2.5)
```

```
print(t)
```

```
('hola', 1, 25.5)
```

Tuplas: Aspectos Particulares

- **Puede un elemento de la tupla ser mutable?**

Consideremos el siguiente caso, donde la tercera componente es una lista.

```
>>> l = [1 , 2 , 3]
>>> t = 'a' , 'b' , l
>>> print(t)
('a', 'b', [1,2,3])
>>> l[1] = 200 # Que pasa con t ?
>>> print( t)
```

En este caso, la tupla guarda una referencia a la lista.

- **Un caso de suma utilidad.** En muchas situaciones, tenemos funciones que queremos que devuelvan **más** de un resultado. La función puede devolver una tupla con los resultados deseados.

```
def min_max_lista(l) :
    return min(l) , max(l)
def main( ) :
    l = [1, 5, 3, 3, 2 , 1, 3, 4]
    minimo , maximo = min_max_lista l)
    t = min_max_lista (l)
```

Tuplas: Resumen

Por qué son esencialmente diferentes de las listas?

- A los fines prácticos funcionan igual, sólo que son inmutables. Solemos usar tuplas en los **returns** de las functions cuando las mismas tienen más de un resultado. Es una cuestión de convención.
- Son estructuras más livianas que las listas.
- Los tipos de datos en su interior suelen variar (a diferencia de las listas, que se espera que los tipos de datos dentro sean homogéneos).
- Las listas se comportan como un conjunto de objetos, las tuplas como un objeto de varias partes (por eso su tamaño es fijo).
- Si cada elemento de la tupla es inmutable; son hasheables: pueden pertenecer a un set y ser key de diccionarios

Tuplas: Probar estos comandos

```
#Una lista y un diccionario vacío
lista_1=[1,2,3]
dic={}

#Una tupla con un elemento mutable, otra con todos sus elementos inmutables
tupla_1=(1,1,lista_1)
tupla_2=(1,1,1)

#qué sucede en estas situaciones?
lista_1.append(4)
tupla_1[2].append(4)
|

#Qué sucede aca?
dic[tupla_1]=33
dic[tupla_2]=33

#Y aca?
tupla_1[0]=3
tupla_2[0]=3
```

Todas las tuplas son inmutables, no todas son hasheables!!

SETS



Sobre los sets

Un set es el equivalente en python de lo que matemáticamente se llama **conjunto**.

- Sus elementos deben ser inhasheables.
- No poseen elementos repetidos
- A diferencia de las tuplas y las listas, no están ordenadas de manera secuencial. Si `set_1` es el nombre de una variable cuyo type es set, decir `set_1[0]` no significa nada. No respetan un orden.
- Poseen una serie de operaciones que en matemática conocemos como **operaciones de conjuntos**.

Sobre los sets

Operaciones:

- ▶ `c = set()`: Crea un conjunto vacío.
- ▶ `c.add(x)`: Agrega el elemento `x` al conjunto `c`.
- ▶ `x in c`: Dice si el elemento `x` está en `c`.
- ▶ `c.remove(x)`: Elimina el elemento `x` de `c`.
Precondición: `x in c` (caso contrario, tira error)
- ▶ `len(c)`: Devuelve la cantidad de elementos de `c`.
- ▶ `c1==c2`: Dice si los dos conjuntos son iguales.
- ▶ `c1 | c2`: Devuelve un nuevo conjunto con $c1 \cup c2$.
- ▶ `c1 & c2`: Devuelve un nuevo conj. con $c1 \cap c2$.
- ▶ `c1 - c2`: Devuelve un nuevo conj. con $c1 \setminus c2$.
- ▶ `list(c)`: Devuelve una lista con los elementos de `c`.

Sobre los sets

```
#Creamos un set y un diccionario vacío
set_1=set([1,2,3])
dic={}

#Qué sucede en cada una de estas operaciones?

set_1.add(4)

set_1.add([1,2,2])

set_1.add(4)

set_1.add((1,1,1))

set_1.add((1,1,[1,2]))

dic[set_1]=1000
```

Los sets son inmutables? Son hasheables?

FOR LOOP



Ciclos for

Si queremos iterar sobre los índices de una lista, podemos utilizar la estructura de control for:

Ejemplo 1: recorrer una lista.

```
l = ['a', 'b', 'c', 'd', 'e']
for i in range(len(l)):
    print(i, l[i])
l = ['a', 'b', 'c', 'd', 'e']
i = 0
while i < len(l):
    print(i, l[i])
    i = i + 1
```

Ejemplo 2: imprimir números pares entre 1 y 20.

```
for i in range(2, 20, 2):
    print(i)
i = 2
while i < 20:
    print(i)
    i = i + 2
```

Ciclos: La función range()

Idea: Al momento de iterar, muchas veces es necesario hacerlo siguiendo progresiones aritméticas con determinados patrones. La función range cumple esta tarea.

range(start, stop, step): Genera una secuencia de números enteros en el rango [start,stop) de *paso* step. Técnicamente no es una lista pero podemos usarlo como si generara una.
start y step son opcionales.

```
>>> range (10) # Toma start = 0 y end = 10. step = 1
>>> range (2,10) # step = 1
>>> range (2,10,3) # Que secuencia genera ?
>>> range (10,2,-1) # Que secuencia genera ?
>>> type (range(10)) # class range
>>> l = list(range(10)) # obtiene la lista de manera explicita
```

For vs while

- Cuál es la principal diferencia de iterar con **for** vs iterar con **while**?
- Los ciclos for necesitan un contador? Necesitan variables que se actualizan?
- La única manera de recorrer un for loop es mediante el uso de range?

Usar for con un alias

- Se le llama sugar syntax a cuando un lenguaje de programación se actualiza y modifica su sintaxis para facilitarle la vida a los devs.
- Dentro de este universo está el alias en los for loops: podés nombrar a los elementos de una estructura como quieras.

```
#Qué hace este código?  
minutos=[3,5,87,4]  
segundos=[]  
for minuto in minutos:  
    segundos.append(minuto*60)
```

- La ventaja es obvia: mayor comodidad y entendimiento del código.
- Se les ocurren **desventajas** de iterar así?

Ejercicio

Ahora una function recibe 2 listas y un set. El set es de los DNIs de alumnos que no rindieron, las 2 listas son los DNI de los alumnos que si rindieron y sus notas.

El output debe ser un diccionario cuya clave sea el DNI del alumno y su valor sea la nota que se sacó en el examen.

OPERACIONES CON LISTAS

Más Listas

- **Concatenación.** El operador `+` permite *concatenar* dos listas, retornando una nueva (sin modificar las listas originales).

```
>>> l 1 = [ 4 , 5 , 6 ]
>>> l 2 = [ 1 , 2 , 3 ]
>>> l 3 = l 1 + l 2 # l 1 y l 2 no se modifican .
[ 4 , 5 , 6 , 1 , 2 , 3 ]
```

Así como la *suma* representa la concatenación, también podemos definir la multiplicación de una lista por un escalar.

```
>>> l 1 = [ 4 , 5 , 6 ]
>>> l 4 = l 1 *3 # Que l i s t a es l 4 ?
```

Si queremos generar una lista con `n` posiciones con un valor por default, **cómo podemos hacer?**

- **Extender una lista.** Sirve para agregar una lista al final de otra.

```
>>> l 1 = [ 4 , 5 , 6 ]
>>> l 2 = [ 1 , 2 , 3 ]
>>> l 2 . extend ( l 1 ) # Como quedan l 2 y l 1 ?
```

Cuál es la principal diferencia entre `+` y `extend`?

Algunas funciones/procedimientos adicionales

- **Sorting** (ordenar). Procedimiento que ordena los elementos de la lista (por ahora, de menor a mayor).

```
>>> l = [ 1 , 4 , 6 , 3 , 2 , 18 , 25 , 23 , 1 ]  
>>> l.sort()  
>>> print(l)
```

- **Operaciones varias.** Python tiene disponibles funciones básicas para aplicar sobre listas.

```
>>> l = [1,4,6,3,2,18,25,23,1]  
>>> sum(l)  
83  
>>> min(l)  
1  
>>> max(l)  
25  
>>> l2 = sorted(l)
```

sorted toma la lista y devuelve una nueva con los mismos elementos, pero ordenados.

Ejercicio

Tenemos una lista con valores numéricos. Implementar una función que calcule la *mediana*.

STRINGS



Strings: Nociones Básicas

Un **string** es una secuencia, ordenada, de *caracteres*. Es similar a tener una lista donde cada posición es un carácter, y que no puede tener elementos de otro tipo. Posee, sin embargo, algunas diferencias con las listas.

```
>>> fruit = 'banana'
>>> name = 'John'
```

Los strings, de forma análoga a las listas, soportan la utilización del operador `[]` para acceder a los elementos. Éstos también están ordenados de 0 a `len(s) - 1`.

```
>>> fruit = 'banana'
>>> print(len(fruit))
6
>>> letter = fruit[1] # Obtenermos 2do caracter
>>> fruit[1] = 'o' # queremos escribir bonano. Se puede?
```

Los índices deben ser enteros y pueden ser expresiones que evalúen a enteros. Los strings son inmutables (al igual que las tuplas). Soportan slicing y operadores como `+` (concatenar) e `in` (pertenencia).

Strings: Nociones Básicas

- **Concatenar.** Dados dos string, el operador + genera uno nuevo con la concatenación de ambos.

```
>>> fruit1 = 'banana'
>>> fruit2 = 'apple'
>>> concat_wospace = fruit1 + fruit2 # Que deberia devolver ?
>>> concat_wspace = fruit1 + ' ' + fruit2 # Que deberia devolver ?
```

Cómo se implementaría algo *análogo* al append de las listas?

- **Comparación.** Al igual que otros tipos de datos, se pueden definir operaciones de comparación sobre las listas.

```
>>> fruit1 = ' banana '
>>> fruit2 = ' apple '
>>> fruit1 == ' apple ' # V o F?
>>> fruit_upper = ' Banana '
>>> fruit_upper < fruit1 # V o F?
```

- **Pertenencia.** El operador in permite verificar si un string está contenido como *subcadena* en otro.
- **Slicing.** Funciona de la misma forma que para las listas.
- Existen muchos otros métodos: upper(), lower(), find(), etc. (ver la documentación)

Iterando strings

Por su estructura, los strings pueden ser *recorridos* de forma análoga a las listas usando while y for.

Qué hace el siguiente código?

```
def ej_string(s):  
    ret = ''  
    for letter in s:  
        ret = letter + ret  
    return ret  
  
def main() :  
    quote = 'YVAN EHT NIOJ'  
    s = ej_string(quote) # Que string tiene s?
```


Ejercicio

Implementar la función `find_char` con el siguiente comportamiento:

- Toma dos parámetros: un string `s` y un caracter (string de longitud 1) `c`.

```
def find_char(s,c):  
    pass
```

- Devuelve un int correspondiente al menor índice de una aparición de `c` en `s`, si `c` está en `s`, y -1 en caso que `c` no esté en `s`. Ejemplos:

```
def main():  
    s = 'banana'  
    pos_n = find_char ( s , 'n ' ) # pos_a = 2 .  
    pos_c = find_char ( s , ' c ' ) # pos_c = -1
```

Duda

Los strings son mutables? Son hasheables?

SOBRE LOS HASHES

Hash

- Venimos diciendo que para que algo sea hasheable debe ser inmutable y todos sus elementos también.
- Además, que algo sea hasheable implica que puede ser la key de un diccionario.

Ahora bien, que es un hash?

- Un hash es una secuencia de caracteres (un string).
- Son generados por un algoritmo complicado que hace las veces de “caja negra”.
- En el caso de las keys de los diccionarios, python hashea las keys para manejarlas con más velocidad a la hora de hacer operaciones.
- Mediante una técnica llamada “hashMap” python busca exactamente la key del diccionario que está buscando, en lugar de repasar las keys una a una.

Algunos algoritmos comunes de hashing son el SHA-256 y el SHA-3

DICCIONARIOS: COMENTARIOS ADICIONALES

Más diccionarios: Claves, significados e items

Algunas operaciones más sobre diccionarios:

- **Longitud.** Llamar a la función *len* con un diccionario como parámetro devuelve la cantidad de items que posee.
- **Obtener las claves.** El método *keys* devuelve las claves.
- **Obtener los significados.** El método *values* devuelve todos los significados.
- **Obtener los items.** El método *items* devuelve todos los ítems. Cada elemento es una tupla donde la primera componente es la clave y la segunda el significado.

```
>>> eng2sp = { 'one': 'uno' , 'two' : 'dos' , 'three' : 'one' }
>>> list(eng2sp.keys())
['one', 'two', 'three']
>>> list(eng2sp.values()) # Notar que incluye repeticiones .
[ 'uno', 'dos', 'uno' ]
>>> list(eng2sp.items())
[('one', 'uno'), ('two', 'dos'), ('three', 'uno')]
```

Notar que el tipo de datos de `dict.keys()`, `dict.values()` o `dict.items()` es uno **endémico del objeto diccionario**. Para hacerlo más “manejable” se lo pasa a lista usando “list”

Ejercicio

Consideren el siguiente diccionario

```
diccionario = { 'primero' : 1 , 'segundo' : 2 , 'tercero' : 3 }
```

Usando las funciones que vimos hasta el momento, escribir un código que imprima la clave y el significado, separados por “ ” y uno por línea.

Aclaración: el código debe ser genérico. No se puede asumir que tiene 3 ítems.

Iterando diccionarios

Existen distintas formas de iterar un diccionario. Escencialmente, con poder acceder a alcanza. Sin embargo, Python provee algo de *syntactic sugar* para que sea mas simple

```
def main():  
    d = {'primero': 1 , 'segundo':  
        2 , 'tercero' : 3}  
    for key in d.keys():  
        print(key,d[key])
```

```
def main():  
    d = {'primero':1 , 'segundo':  
        2 , 'tercero': 3}  
    for key in d:  
        print (key, d[key])
```

Otra alternativa podría ser iterar las *tuplas* de items del diccionario.

```
def main():  
    d = {'primero':1, 'segundo':  
        2, 'tercero': 3}  
    for t in d.items():  
        print(t[0], t[1])
```

```
def main():  
    d = { 'primero': 1 , 'segundo':  
        2, 'tercero': 3}  
    for key , val in d :  
        print key, val)
```